

龙芯平台 BSP 设计报告

文档版本：v1.3

适用平台：龙芯（Loongson）处理器系列（LoongArch 架构）

关键技术选型：Linux-RT 实时内核 · PMON 引导 · ACPI 设备管理

文档性质：板级支持包（Board Support Package）设计说明

编制日期：2025年

目录

- 1. [概述](#)
 - 1.1 [编写目的](#)
 - 1.2 [背景与术语](#)
 - 1.3 [适用范围](#)
- 2. [开发环境](#)
 - 2.1 [宿主系统（Ubuntu）](#)
 - 2.2 [交叉编译工具链安装](#)
 - 2.3 [环境变量配置](#)
 - 2.4 [工具链验证](#)
- 3. [BSP 总体架构](#)
 - 3.1 [层次模型](#)
 - 3.2 [软件交付物清单](#)
- 4. [硬件平台](#)
- 5. [软件组成与职责](#)
 - 5.1 [Bootloader（PMON）](#)
 - 5.1.1 [PMON 设计开发要点](#)
 - 5.1.2 [PMON 引导与镜像更新分层设计](#)
 - 5.1.3 [SPI-NOR Flash 分区规划](#)
 - 5.1.4 [ACPI 设备管理（替代设备树）](#)
 - 5.2 [操作系统内核（Linux-RT）](#)
 - 5.3 [文件系统](#)
 - 5.3.1 [文件系统命令集](#)
 - 5.3.2 [启动 系统配置](#)
 - 5.3.3 [文件系统选型](#)
- 6. [系统启动流程](#)

- [6.1 启动流程图](#)
- [6.2 PMON 引导与烧写命令示例](#)
- [7. 关键驱动模块设计](#)
 - [7.1 时钟与电源管理](#)
 - [7.2 串口 \(UART\)](#)
 - [7.3 存储控制器 \(NAND/eMMC/SATA\)](#)
 - [7.4 网络 \(GMAC\)](#)
 - [7.5 显示与 GPU](#)
 - [7.6 PCIe 与中断](#)
 - [7.7 设计开发的关键板级驱动](#)
 - [7.7.1 基于 PCIe+FPGA 的平台接口扩展与控制驱动](#)
 - [7.7.2 CANFD 驱动](#)
 - [7.7.3 网口驱动 \(GMAC ×3, TSN\)](#)
 - [7.7.4 扩展串口驱动](#)
- [8. 移植与适配要点](#)
- [9. 构建与烧录系统](#)
- [10. 测试与验证](#)
- [11. 常见问题排查](#)
- [12. 附录](#)

1. 概述

1.1 编写目的

本文档描述基于龙芯（Loongson）处理器的**板级支持包（BSP，Board Support Package）**设计方案，覆盖从硬件上电、Bootloader、操作系统内核到设备驱动与根文件系统的完整软件栈。目的是为移植工程师、驱动开发者与系统集成人员提供统一的架构说明与实现指引，确保 BSP 在不同单板（Board）间具备可复用、可裁剪、可验证的特性。

1.2 背景与术语

术语	全称	说明
BSP	Board Support Package	介于硬件与操作系统之间的适配层
LoongArch	Loongson Architecture	龙芯自主指令集架构（LA32/LA64...）
PMON	—	龙芯传统 Bootloader（类 BIOS），本方案指定引导固件

术语	全称	说明
ACPI	Advanced Configuration & Power Interface	高级配置与电源接口，本方案的设备管理机制
RSDP	Root System Description Pointer	根系统描述指针，PMON 传给内核的 ACPI 入口
AML/DSDT	ACPI Machine Language / Differentiated System Description Table	ACPI 机器语言 / 主板专属设备表
RT	Real-Time	实时（Linux-RT / PREEMPT_RT 内核）
GMAC	Gigabit Media Access Controller	千兆以太网控制器
SMP	Symmetric Multi-Processing	对称多处理

1.3 适用范围

- 本文 BSP 以龙芯 2K2000（2× LA364 核，LoongArch）为具体实现对象，硬件细节参见第 4 章（依据《龙芯 2K2000 处理器用户手册 V1.0》）。
- 龙芯 3A/3B/3C（服务器/桌面，LoongArch）系列单板（架构同源，ACPI 表与驱动可复用）。
- 龙芯 2K/2H（嵌入式，LoongArch）系列单板。
- 兼容 MIPS 架构的早期 3A1000/3A2000 等型号（降级适配）。

2. 开发环境

本 BSP 的构建与移植工作均在 **Ubuntu x86_64** 宿主系统上完成，使用龙芯官方 GNU 交叉编译工具链对 2K2000（LoongArch）目标平台进行交叉编译。本章给出从零搭建可复现构建环境的完整步骤（亦可配合《2K2000编译环境搭建.ppt》演示材料使用）。

2.1 宿主系统（Ubuntu）

项目	推荐配置
发行版	Ubuntu 20.04 LTS / 22.04 LTS（x86_64）
CPU	4 核及以上
内存	≥ 8 GB（内核与根文件系统并行编译更顺畅）
磁盘	≥ 50 GB 空闲（源码 + 中间产物 + 根文件系统）
网络	可访问龙芯开源社区 / 内部 2K2000 BSP 仓库

2.2 交叉编译工具链安装

使用龙芯官方发布的 GNU 工具链：

```
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.5.tar.xz
```

该工具链基于 **GCC 8.3**，目标前缀为 `loongarch64-linux-gnu-`，支持 LoongArch LA32/LA64 指令集，是 2K2000 等嵌入式平台验证版本（rc1.5）。

安装步骤：

```
# 1. 获取工具链（龙芯开源社区或 2K2000 BSP 交付包）
# 示例：wget <下载地址>/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-
gnu-rc1.5.tar.xz
wget <工具链下载地址>/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-
rc1.5.tar.xz

# 2. 解压到 /opt
sudo tar -xJf loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-
rc1.5.tar.xz -C /opt/

# 解压后目录：/opt/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-
rc1.5/
```

说明：升级工具链时只需替换该目录并更新 `PATH`，无需改动任何构建脚本。

2.3 环境变量配置

将工具链 `bin` 加入 `PATH`，并设好交叉编译前缀，供 PMON、内核、根文件系统统一使用：

```
cat >> ~/.bashrc <<'EOF'
export TOOLCHAIN=/opt/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-
rc1.5
export PATH=$TOOLCHAIN/bin:$PATH
export ARCH=loongarch
export CROSS_COMPILE=loongarch64-linux-gnu-
EOF
source ~/.bashrc
```

关键变量说明：

变量	取值	作用
ARCH	loongarch	内核/驱动构建目标架构
CROSS_COMPILE	loongarch64-linux-gnu-	交叉编译器前缀
PATH	含工具链 bin	使 loongarch64-linux-gnu-gcc 可直接调用

2.4 工具链验证

```
loongarch64-linux-gnu-gcc --version
# 期望输出包含: gcc version 8.3.0 (Loongson GNU toolchain rc1.5) ...

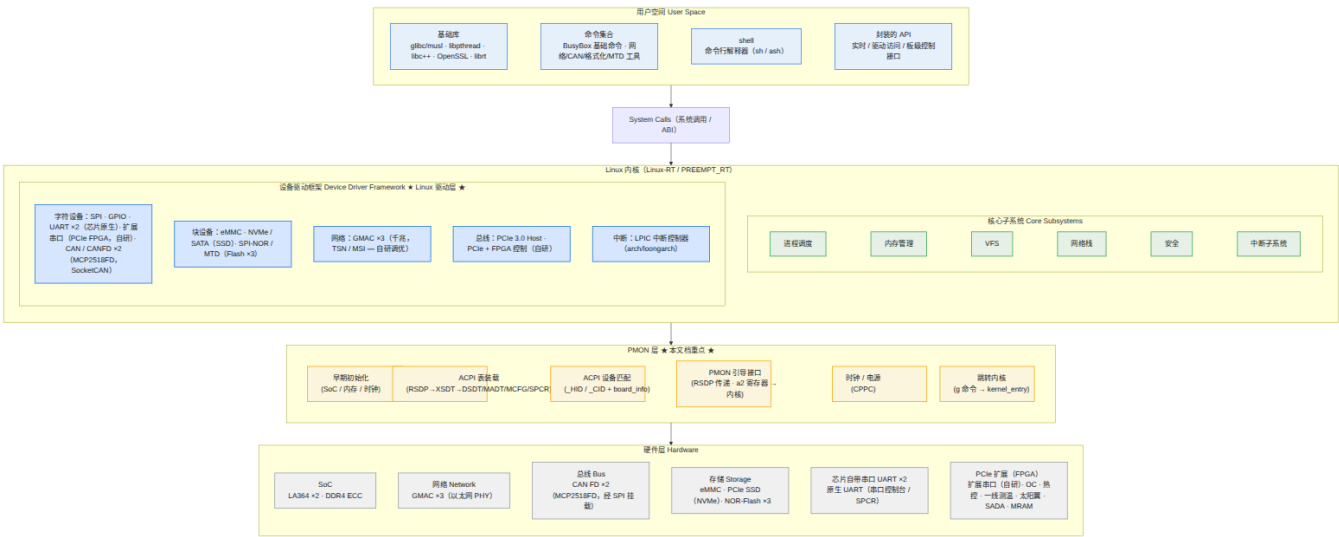
# 交叉编译最小测试程序
echo 'int main(){return 0;}' > test.c
loongarch64-linux-gnu-gcc test.c -o test
file test
# 期望: test: ELF 64-bit LSB executable, LoongArch
```

若 file 显示目标架构为 **LoongArch**，说明工具链配置正确，可用于后续 PMON / 内核编译。

3. BSP 总体架构

3.1 层次模型

BSP 以"硬件抽象 + ACPI 固件描述"为核心，基于 **PMON 引导 + Linux-RT 实时内核 + ACPI 设备管理机制** 构建。整体采用经典的分层栈结构，自上而下为：**用户空间 → Linux 内核（核心子系统 + 设备驱动框架） → PMON 层 → 硬件层**。"Linux 驱动层"以内核内的 **设备驱动框架（Device Driver Framework）** 显式体现，并按字符 / 块 / 网络 / 总线 / 中断五类子系统组织，明确标注本项目自研驱动（见第 7 章）。



自顶向下依赖：用户空间经 **系统调用** 进入 Linux 内核；内核核心提供调度 / 内存 / VFS / 网络等能力，**设备驱动框架** 即显式体现的 Linux 驱动层，向下经 **PMON 层**（金色高亮，本文档重点）完成早期初始化、ACPI 表装载 / 匹配与引导接口，最终驱动硬件层。图示为导出的静态图片（层次模型.png / PMON分层设计.png），可在任意 Markdown 预览中直接查看，不再依赖 Mermaid 实时渲染。

设计原则：

- 1. **板级差异收敛到 ACPI**：单板硬件拓扑由 ACPI 表（DSDT/SSDT/MCFG/MADT 等）描述，内核代码与 SoC 通用，板级差异仅通过固件表区分。
- 2. **驱动与板级解耦**：Linux 驱动层按字符/块/网络/总线/中断五类子系统划分，驱动以 ACPI _HID / _CID 匹配方式绑定（替代 Device Tree 的 compatible），并以 board_info 补充非 ACPI 设备，便于跨板复用。
- 3. **实时性保障**：Linux-RT（PREEMPT_RT）作用于内核核心与驱动层，关键中断线程化（threadirqs）、临界区可抢占（rt_mutex + 优先级继承）、HIGH_RES_TIMERS 提供高精度定时，满足确定性时延要求。
- 4. **自研驱动内聚**：PCIe+FPGA 控制、CANFD（SocketCAN）、GMAC ×3（TSN，其中 2 口实网调优）、扩展串口等自研驱动归属 Linux 驱动层对应子系统，不侵入 PMON 层与内核核心，降低维护耦合。
- 5. **可裁剪**：通过 Kconfig / 编译开关裁剪不必要的子系统与驱动，满足资源受限场景。

3.2 软件交付物清单

类别	交付物	格式
Bootloader	pmon.bin	二进制（含 ACPI 表装载逻辑）
内核	vmlinux / Image (Linux-RT，集成 PREEMPT_RT)	二进制

类别	交付物	格式
根文件系统	rootfs.tar.gz / rootfs.ext4 （含 RT 调优）	镜像
驱动模块	*.ko （含本项目开发：PCIe+FPGA 控制、CANFD、扩展串口、GMAC ×3）	内核模块
文档	本报告	Markdown

4. 硬件平台

本 BSP 的硬件平台为**增强型综合电子核心管理卡**，主控为龙芯 2K2000（2× LA364 / LoongArch）。

- **芯片手册**：SoC 寄存器与模块级细节，详见 《**龙芯2K2000处理器用户手册_V1.0 试用版.pdf**》。
- **硬件原理图**：板级接口、连接器、外设与电源 / 时钟布局，详见 《**增强型综合电子核心管理卡硬件原理图**》。

5. 软件组成与职责

5.1 Bootloader（PMON）

本方案统一采用 **PMON** 作为引导固件（不依赖 U-Boot）。PMON 在龙芯平台上承担"类 BIOS"职责，并提供 ACPI 运行环境所需的固件表装载能力。

职责：

1. 最早期硬件初始化（CPU、缓存、内存控制器训练）。
2. 构造并装载 **ACPI 表**（将 DSDT/SSDT/MCFG/MADT 等以 RSDP 指针形式放置在合法内存区域）。
3. 加载内核镜像到内存，并通过 a2 寄存器向内核传递 **RSDP 物理地址**。
4. 设置启动参数（cmdline），跳转内核入口。
5. 提供**串口控制台**（早期排错与交互），并支持多种**存储器启动介质**加载内核与根文件系统：
 - **SPI-NOR Flash**：PMON 自身与 ACPI 表常驻其中，可直接从 Flash 读取内核镜像启动；
 - **NVMe（SSD，PCIe 接口）**：从 PCIe 连接的 NVMe 固态硬盘读取内核与根文件系统；
 - **SATA（硬盘）**：从 SATA 连接的机械/固态硬盘读取内核与根文件系统；
 - **eMMC**：从板载 eMMC 分区读取内核与根文件系统。

5.1.1 PMON 设计开发要点

本 BSP 在 PMON 固件层设计开发并支持以下核心能力（引导/交接等后续步骤见第 6 章启动流程）：

- **大容量 SPI-NOR Flash 读写驱动支持**：驱动支持大容量（>16MB）SPI-NOR Flash，采用 4-byte 地址模式与 QE 位使能，提供扇区擦除 / 页编程 / 读；向上封装 `flash_read()` / `flash_write()` / `flash_erase()` 统一接口（见 5.1.2 分层设计），PMON、ACPI 表与镜像常驻其中。
- **镜像打包与分区策略**：采用 **rootfs 与 kernel 打包为单一系统镜像、整体压缩（约 11MB）** 的策略；Flash 分区呈现为 **boot 驱动区（raw）、系统区（raw）、空洞区（保留）、APP1~APP3 应用区（JFFS2）** 四部分（详见 5.1.3）。
- **镜像更新指令（fload / dfu）**：提供两条升级指令——`fload` 用于刷写 **PMON**（写入 boot 驱动区）；`dfu` 用于升级 **操作系统**，按系统升级方式将整体系统镜像**整体刷写**至系统区（含完整性校验），无需离线编程器。
- **内核格式解析**：解析内核镜像格式（ELF / 压缩 / 扁平 Image、与 initramfs 整合形态），定位入口与必要段。

5.1.2 PMON 引导与镜像更新分层设计

为清晰表达 5.1.1 各能力在固件内的实现结构，将 PMON 的 SPI-NOR Flash 引导/更新路径设计为**自底向上的八层**，每层仅向上提供明确接口、屏蔽下层细节（SPI 控制器驱动位于最底层）：

① SPI 控制器驱动 (SPI 读写接口)

spi_transfer · 4-byte 地址 ·
QE 位



② Flash 块驱动 (读写接口)

flash_read / flash_write /
flash_erase



③ 分区解析

映射 boot / 系统 / 空洞 /
APP 分区

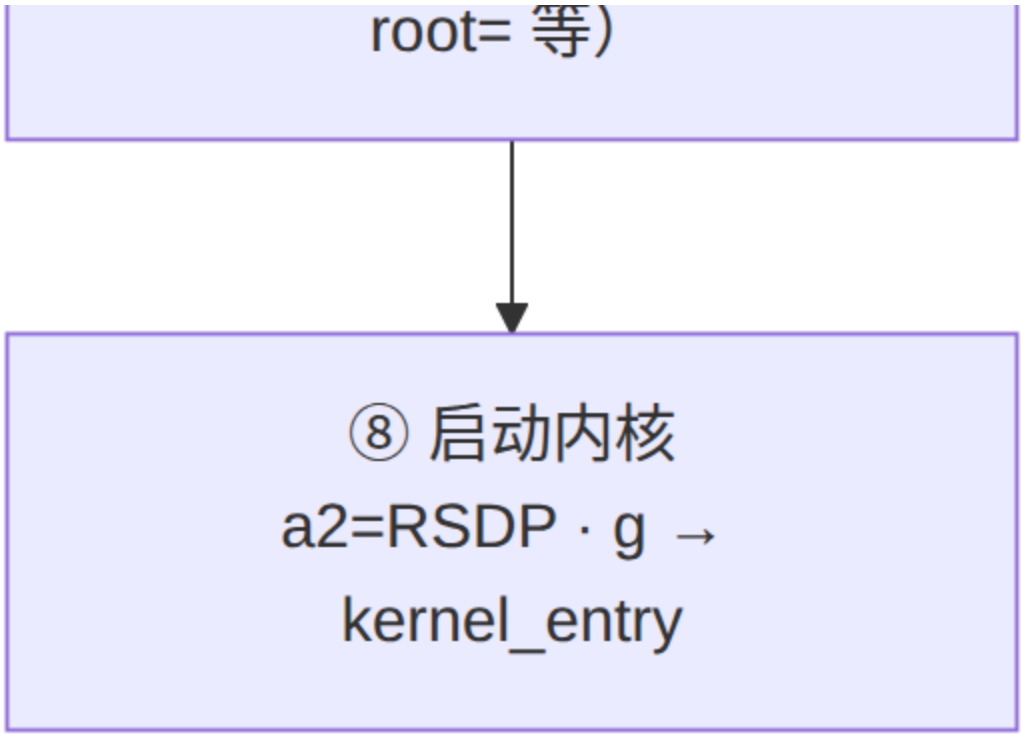


④ 装载校验
解包镜像 · CRC32 /
SHA256 / 签名

⑤ 解析操作系统入口
定位 ELF / Image 入口地址

⑥ 传递设备属性
ACPI _HID/_CID +
board_info → 内核

⑦ 启动参数
构造 cmdline (console= /



层	名称	职责与关键接口
①	SPI 控制器驱动（SPI 读写接口）	操作 SPI 控制器，提供 spi_transfer() 等 SPI 读写接口（时钟 / 片选 / 模式 / 速率）；支持 4-byte 地址与 QE 位以适配大容量 Flash
②	Flash 块驱动（读写接口）	在 SPI 之上封装 NOR Flash 协议（Read / Page Program / Sector Erase、Status/WEL/WRSR），提供统一 flash_read() / flash_write() / flash_erase() 块读写接口
③	分区解析	解析 Flash 内部分区表，划分为 boot 驱动区（raw）、系统区（raw）、空洞区（raw, 保留）、APP1~APP3（JFFS2） ，映射各分区起始与长度
④	装载校验	从系统区解包整体压缩镜像（rootfs+kernel 组合，约 11MB），并对镜像做 CRC32 / SHA256 / 签名校验，防止损坏或非法镜像被引导
⑤	解析操作系统入口	解析镜像内 ELF / Image / initramfs / ACPI 表封装，按 Program Header 搬移 PT_LOAD 段并 定位操作系统入口地址
⑥	传递设备属性	整理板级设备属性（ACPI _HID / _CID + board_info、硬件拓扑），在引导阶段传递给内核，供驱动匹配与设备枚举
⑦	启动参数	构造内核启动参数 cmdline（console= / root= 等），准备 RSDP 等运行环境
⑧	启动内核	通过 a2 传 RSDP、执行 g 跳转 kernel_entry，完成引导交接

5.1.3 SPI-NOR Flash 分区规划

基于 5.1.1 的"boot 驱动区 + 系统区 + 应用区"三层划分，结合 2K2000 大容量 SPI-NOR Flash，具体分区规划如下（偏移为示意，按实际 Flash 容量调整）：

分区	大小	偏移（示意）	格式	内容 / 用途
boot 区	2MB	0x0000000	raw	PMON + SPI-NOR 驱动 + ACPI 表； fload 刷写目标
操作系统分区	16MB	0x2000000	raw	压缩的 rootfs+kernel 系统镜像（约 11MB）； dfu 整体刷写目标
空洞区	2MB	0x1200000	raw	保留空洞（对齐 / 安全余量），不存放业务数据
APP1	按需求	0x1400000 起	JFFS2	应用 1（镜像 / 数据）
APP2	按需求	紧随 APP1	JFFS2	应用 2（镜像 / 数据）
APP3	按需求	紧随 APP2	JFFS2	应用 3（镜像 / 数据）

说明：boot 区与操作系统分区为 **raw** 裸镜像（直接 fload / dfu 刷写），大小固定；**空洞区（2MB, raw）** 作为保留对齐 / 安全余量，不存放业务数据；APP1/APP2/APP3 为 **JFFS2** 文件系统，可在运行时挂载读写，按实际业务需求在剩余空间内依次分配。升级时 fload 仅更新 boot 区、dfu 仅整体刷写操作系统分区，应用区互不干扰。

5.1.4 ACPI 设备管理（替代设备树）

本方案**不使用 Device Tree**，板级硬件拓扑完全由 **ACPI 表**描述。设备枚举、资源分配（MMIO/IRQ/DMA）、电源与中断信息均来自 ACPI 命名空间，PMON 在引导阶段装载这些表，内核在 early boot 阶段解析。ACPI 表结构与命名空间规范详见 《**ACPI_Spec_xx.pdf**》（ACPI Specification）。

ACPI 表组成：

表	作用
RSDP	根系统描述指针，PMON 传递给内核（ a2 寄存器）
XSDT	指向各子表的入口数组
FADT	固定 ACPI 描述表（电源/睡眠/复位寄存器）
DSDT	差异系统描述表（主板专属 AML 设备定义）
MADT	多 APIC 描述表（CPU/中断控制器/LPIC 拓扑）

表	作用
MCFG	PCIe 增强配置空间 MMIO 基址窗口
SPCR	串口控制台（早期 console=ttyS0 自动定位）

典型 ASL（DSDT 片段）示例：

```

DefinitionBlock ("", "DSDT", 2, "LOONSON", "LS3A5000", 0x00000001)
{
    Scope (\_SB)
    {
        Device (UART0) {
            Name (_HID, "LOON0001")           // 龙芯 UART 硬件 ID
            Name (_UID, 0x0)
            Name (_CRS, ResourceTemplate () {
                Memory32Fixed (ReadWrite, 0x1fe001e0, 0x100)
                Interrupt (ResourceConsumer, Level, ActiveHigh, Exclusive) { 2
            }
        })
        Method (_STA) { Return (0x0F) }
    }

    Device (GPCI) {                          // PCIe Root Complex
        Name (_HID, "PNP0A08")
        Name (_CID, "PNP0A03")
        Name (_CRS, ResourceTemplate () {
            QWordMemory (ResourceConsumer, , MinFixed, MaxFixed,
                NonCacheable, ReadWrite, 0, 0x40000000, 0x7fffffff, , ,
WMEM)
        })
        Name (_CCA, 1)
    }
}

```

板级差异仅体现在 DSDT/SSDT（通过 `iasl` 编译为 AML），内核无需为不同单板修改 C 代码，符合“板级收敛到固件”的设计原则。

5.2 操作系统内核（Linux-RT）

- **内核基线：**基于官方提供的 `linux_4.19.190.8.24.orig.tar.gz` 源码，打上对应的 **PREEMPT_RT (RT) 补丁**（4.19 系列 rt 补丁）后，作为本 BSP 的实时内核基线（Linux-RT）；该基线已集成 LoongArch 支持与 2K2000 相关适配。

- **关键子系统：**本 BSP 重点适配的板级外设与子系统包括 **SPI、GPIO、PCIe、中断控制器、串口 (UART)、eMMC、SSD (NVMe)、SPI-NOR Flash、以太网 (GMAC)**。
- **设备枚举基于 ACPI + board_info：**以 ACPI 命名空间为主，辅以 board_info 描述 ACPI 未能覆盖的板级固定设备；驱动以 _HID / _CID 匹配（替代 Device Tree 的 compatible）。
- **实时性相关 (PREEMPT_RT)：**
 - **全内核可抢占：**PREEMPT_RT 将多数自旋锁改写为可休眠的 rt_mutex，关抢占 / 关中断的临界区大幅缩小，中断与多数内核代码路径均可被高优先级任务抢占。
 - **中断线程化：**通过 threadirqs (CONFIG_IRQ_FORCED_THREADING) 将硬件中断搬移到内核线程 (irq/x)，不再长时间关中断；与第 7 章驱动的中断线程化 (PCIe+FPGA / CANFD / GMAC) 协同降低抖动。
 - **高精度定时器：**CONFIG_HIGH_RES_TIMERS 提供 μs 级定时分辨率，满足确定性周期任务。
 - **优先级继承：**rt_mutex 支持优先级继承，消除优先级反转，保障实时任务时限。
 - **实时调度与隔离：**实时任务采用 SCHED_FIFO / SCHED_RR；可用 isolcpus / nohz_full (CONFIG_NO_HZ_FULL) 隔离 CPU、关闭周期 tick，进一步降低抖动。
 - **内存确定性：**关键任务以 mlock() / mlockall() 锁定内存，避免页错误与换页引入的不可预期延迟。

5.3 文件系统

本 BSP 的根文件系统采用 **Buildroot** 构建最小可启动根文件系统 (ramdisk 形式，见 5.3.2)，并预置下列命令与工具以满足板级调试与现场维护。

5.3.1 文件系统命令集

类型	类别	命令	说明
BusyBox 基础命令	基础命令集	ls / cd / cat / cp / mv / rm / mkdir / ps / top / free / uname / dmesg / echo / grep / sed / awk / sh / mount / umount / df	BusyBox 集成的大量基础命令（文件/目录、系统信息、文本与 Shell、挂载与存储等），构成最小可用 shell 环境
特别指令	网络	ifconfig / ip / ping / iperf3 / ethtool / tftp	网络联通性、带宽与网卡参数调试 (GMAC 见 7.7.3)
特别指令	CAN 测试	candump / cansend / cangen	CAN / CAN FD 报文收发与回环测试 (见 7.7.2)
特别指令	文件系统格式化	mkfs.ext4 / mkfs.jffs2	根文件系统与应用区 (JFFS2) 镜像生成
特别指令	MTD 工具	mtddinfo / mtd_debug / flash_erase / flashcp	SPI-NOR 等 MTD 设备访问、擦除与写入 (不含 NAND 工具)

注：BusyBox 集成大量基础命令（文件 / 系统 / 文本 / 挂载等），下表将其单独归为「基础命令」类型；其余 BSP 专用工具归为「特别指令」。

说明：应用区 APP1~APP3 为 JFFS2（见 5.1.3），可在运行时以 `flash_erase + mkfs.jffs2 + flashcp` 预置镜像，或直接挂载读写。

5.3.2 启动 系统配置

根文件系统为 **ramdisk (initramfs)**，并在 Buildroot 阶段**尽可能裁剪至最小**；系统管理采用**最简单的 init**（如 BusyBox `init`），以减少文件体积、加快启动。其启动与挂载流程如下：

- 1. **挂载根文件系统**：内核加载 ramdisk，将其挂载为根文件系统（/）。
- 2. **执行 init 程序**：运行最简 `init`，由其拉起系统初始化脚本 `rcS`。
- 3. **挂载 JFFS2（应用存储）**：`rcS` 中挂载 SPI-NOR Flash 应用区（APP1~APP3）的 **JFFS2** 文件系统，用于应用存储（小文件频繁读写，见 5.3.3）。
- 4. **挂载 ext4（大文件存储）**：挂载 eMMC / SATA(硬盘) / NVMe(SSD) 上的 **ext4** 文件系统，用于大文件持久化。
- 5. **拉起 autostart.sh**：`rcS` 触发 `autostart.sh` 脚本；业务程序的其它自启动项均配置在 `autostart.sh` 中，便于现场维护。

JFFS2 与 ext4 均挂载于 `/mnt` 目录下（如 `/mnt/app` 为 JFFS2、`/mnt/data` 为 ext4）。上述加载与挂载对应用开发者透明——业务程序直接访问已挂载目录，无需关心底层介质与挂载细节。

5.3.3 文件系统选型

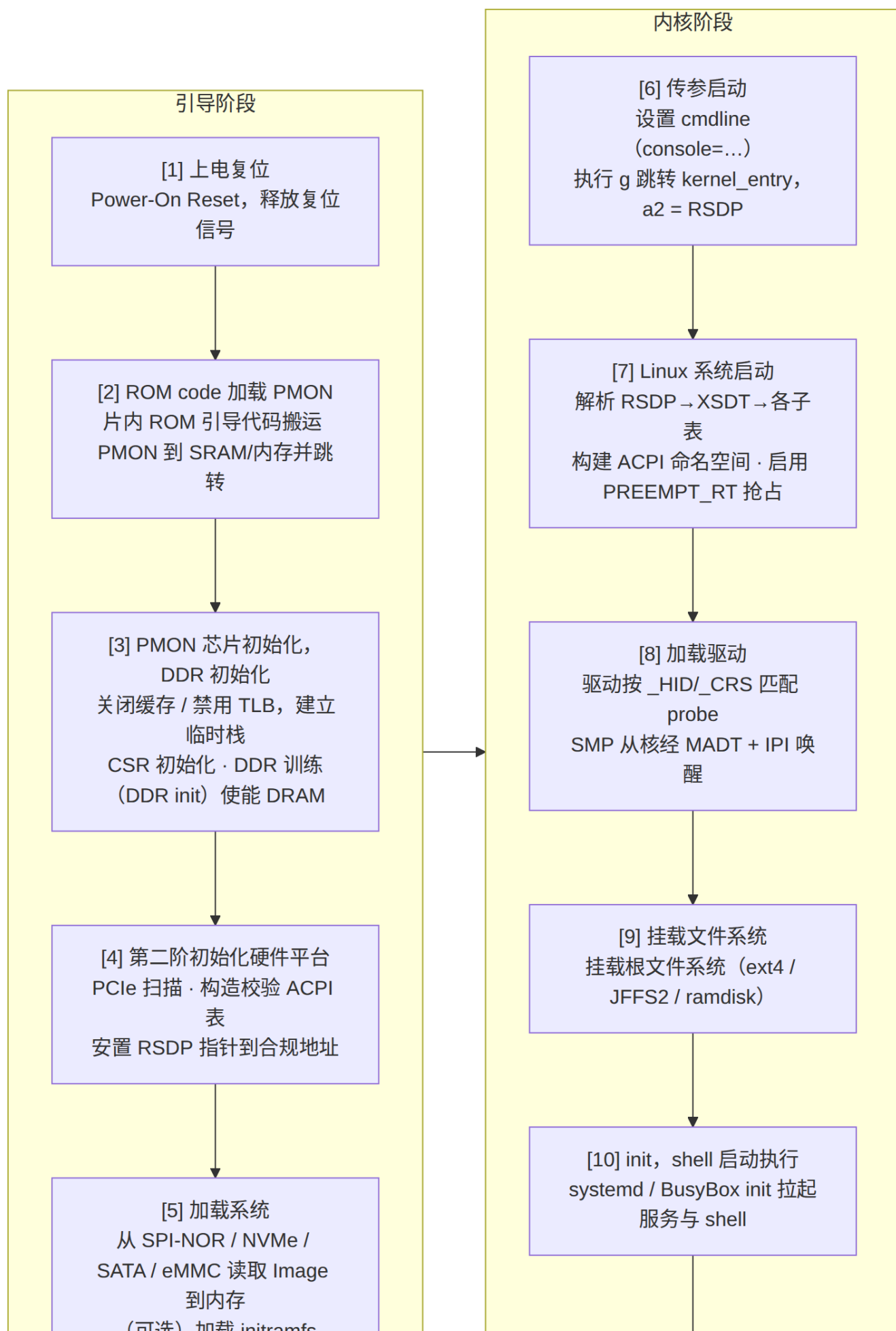
本 BSP 涉及三类文件系统，按承载介质与业务诉求选型如下：

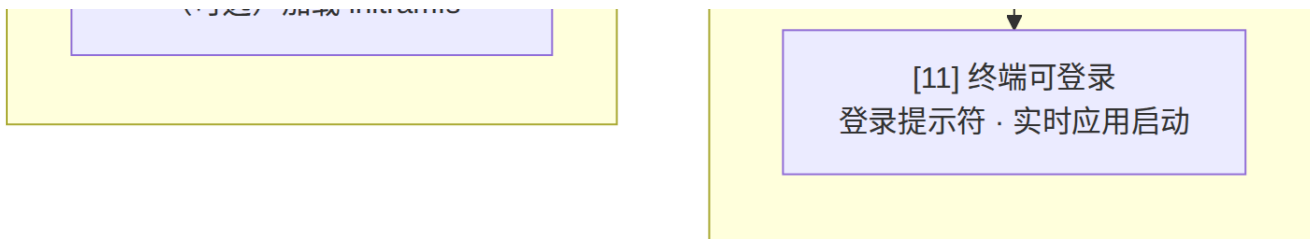
文件系统	承载介质	持久化	选型原因
ramdisk (initramfs)	SPI-NOR 系统区（加载入内存）	否（内存态，启动后交权给持久化存储）	全内存运行、免块设备访问；镜像裁剪至最小后可极快加载 / 解包，显著缩短启动时间，且不引入块设备延迟影响实时性
ext4	eMMC / SATA(硬盘) / NVMe(SSD)	是	成熟稳定、支持大容量与日志掉电保护，适合存放应用数据、日志等需持久化且读写频繁的内容
JFFS2	SPI-NOR Flash 应用区（APP1~APP3）	是	专为 NOR Flash 设计，实现简单、具备磨损均衡与断电保护，提供比裸 MTD 更完善的文件管理，适合应用区小文件频繁读写

6. 系统启动流程

6.1 启动流程图

下图给出从片上电到用户态登录的完整引导链路（图片见 [启动流程图.png](#)）。左列：PMON 引导阶段；右列：Linux 启动阶段。两列之间由 "加载系统 → 传参启动" 完成固件到内核的切换：





本 BSP 统一采用 PMON 引导（不依赖 U-Boot）：PMON 承担早期硬件初始化、ACPI 表装载与内核加载跳转；内核经 a2 收到的 RSDP 完成设备枚举，无需设备树。

关键节点说明：

- **DDR 训练 ([3])**：若内存参数错误，系统会在 [3] 阶段死机，需核对 SPD 或手动配置时序。
- **RSDP 传递 ([6])**：PMON 必须把 RSDP 放在合规地址（通常低端内存），并通过 a2 传给内核；内核校验 RSDP signature ("RSD PTR ") 与 checksum，失败则无法枚举设备。
- **ACPI 命名空间 ([7]→[8])**：acpi_ns_build 解析 DSDT/SSDT，驱动通过 acpi_match_device() 按 _HID 绑定，无需设备树。
- **SMP 启动 ([8])**：从核由主核经 MADT 描述的 CPU 拓扑 + IPI 唤醒，需正确实现 smp_boot_secondary。

6.2 PMON 引导与烧写命令示例

6.2.1 PMON 烧写 (fload)

将 PMON 固件镜像（gzrom.bin）写入 SPI Flash：

```
fload /dev/fs/ext3@wd0b/gzrom.bin
```

6.2.2 内核更新与启动 (dfu / load)

通过 dfu 更新内核镜像：

```
dfu /dev/fs/ext3@wd0b/vmlinux
```

当根文件系统位于 ext3 分区（设备 wd0b）时，依次加载内核与 initrd，并通过 g 携带启动参数跳转：

```
load /dev/fs/ext3@wd0b/vmlinux-4.19.0-19-loongson-3
initrd /dev/fs/ext3@wd0b/initrd.img-4.19.0-19-loongson-3
g console=ttyS0,115200 earlycon=uart,mmio,0x1fe001e0 loglevel=8
```

7. 关键驱动模块设计

7.1 时钟与电源管理

- 时钟源来自 SoC 内部 PLL，通过 `clk-loongson` 驱动管理分频。
- 电源管理：实现 `cpuidle`、`cpufreq`，基于 **ACPI CPPC / _PSS** 对象（由 DSDT 提供 P-state/C-state 表），或由 PMON 固件暴露的私有接口。
- 设计要点：休眠（suspend）经 ACPI FADT 的睡眠/唤醒寄存器，唤醒后恢复 PLL 与内存控制器状态；实时场景下应谨慎使用 deep idle，避免破坏确定性时延。

7.2 串口（UART）

- 早期调试依赖串口，是"BSP 第一驱动"。
- 实现 8250 兼容驱动或龙芯专用 UART 驱动，资源由 **ACPI _CRS** 提供（Memory32Fixed 描述 MMIO、Interrupt 描述 IRQ），驱动经 `_HID`（如 `LOON0001`）匹配。
- 建议通过 **SPCR 表**自动定位控制台串口，使 `console=ttyS0` 无需在 cmdline 显式指定；波特率在内核与 PMON 中保持一致（通常 115200）。

7.3 存储控制器（NAND/eMMC/SATA）

存储	驱动路径	备注
SPI NOR	<code>spi-nor</code>	存放 PMON 与 ACPI 表镜像
eMMC	<code>mmc</code> 子系统	根文件系统常见载体
SATA	<code>ahci</code> / <code>libata</code>	大容量存储
NVMe	<code>nvme over PCIe</code>	高速盘

注意：eMMC 分区方案需与 Bootloader 烧录脚本一致，避免内核无法挂载 root。

7.4 网络（GMAC）

- 驱动：`stmmac`（Synopsys MAC）或板载 RTL PHY，通过 ACPI `_HID` 匹配（`acpi_match_device`）。
- 网络资源由 **ACPI DSDT** 描述：MAC 寄存器窗口（`_CRS` MMIO）、中断、MDIO 子设备与 `phy-mode`（`rgmii/txid`）通过 `_DSD` 属性提供。
- 本项目开发要点（GMAC ×3 网口，其中 2 口 TSN 调优）：见 7.7.3。验证：`ifconfig up` → `ping` 网关；关注 RGMII 延时（tx/rx delay）配置。

7.5 显示与 GPU

- 龙芯集成显示控制器（DC），驱动支持 HDMI/DVI 输出与 framebuffer。

- 独立显卡走 PCIe，依赖 `radeon / amdgpu` 等上游驱动 + 固件。
- 最小系统可先禁用显示，以串口作为唯一控制台，降低移植复杂度。

7.6 PCIe 与中断

- Root Complex 驱动枚举下游设备，分配 `bus/device/function` 与 MMIO 窗口。
- 中断：LA 架构使用 `lpic` 控制器，MSI/MSI-X 需正确映射。
- 常见问题：PCIe 设备无法枚举 → 检查 `PERST#` 复位时序与参考时钟。

7.7 设计开发的关键板级驱动

7.1–7.6 为 2K2000 SoC 内置模块的**基线驱动**；本节汇总**本项目在 BSP 中实际设计开发**的四类板级增值驱动，是单板差异化能力的核心交付。所有驱动均遵循 **ACPI _HID 绑定** 与 **Linux-RT 实时化（中断线程化）** 约束。

7.7.1 基于 PCIe + FPGA 的平台接口扩展与控制驱动

- **背景**：2K2000 原生 I/O 资源有限，通过 PCIe 连接 FPGA (RC↔EP) 扩展 GPIO、ADC/DAC、SPI、I2C、定时器、数字量 I/O 及专用控制逻辑，构成"平台扩展控制卡"。
- **驱动框架**：字符设备 (`misc / cdev`) + `sysfs / ioctl` 控制面；FPGA 作为 PCIe Endpoint 挂载到 2K2000 的 PCIe Root Complex。
- **核心实现**：
 - `probe` 阶段按 **vendor/device ID 或 ACPI _HID** 匹配，使能 PCIe 设备、请求并映射 **BAR (MMIO)** 到内核虚址，建立寄存器访问抽象层；
 - 实现 `read / write / unlocked_ioctl`，将应用层对扩展接口（引脚、寄存器、控制字）的访问翻译为 BAR 读写；
 - **中断**：FPGA 经 **MSI/MSI-X** 或 `INTx` 上报事件，驱动注册**线程化中断** (`request_threaded_irq`)，满足 Linux-RT 低抖动要求；
 - **DMA (可选)**：流式 DMA 缓冲区映射 (`dma_map_single / dma_alloc_coherent`)，并依赖 2K2000 SCache 的**目录协议**维护 DMA 与 CPU Cache 一致性（细节见上传的《龙芯2K2000处理器用户手册_V1.0 试用版.pdf》）；
 - 与 ACPI 集成：在 DSDT 中声明该 PCIe 设备 (`_HID / _CRS / Interrupt`)，使内核自动 `probe`。
- **关键文件 (示例)**：`drivers/misc/ls2k2000_fpga_ctrl.c`、`include/uapi/linux/ls_fpga.h`。

7.7.2 CANFD 驱动

- 2K2000 片内集成 **6× CAN 控制器**（资源与寄存器细节见上传的《龙芯2K2000处理器用户手册_V1.0 试用版.pdf》）；本项目基于 **SocketCAN** 框架实现/适配 **CAN FD** 驱动（经典 CAN 2.0B + FD，数据段最高约 8 Mbps）。

- **核心实现：**
 - 通过 **ACPI _HID** 绑定每路 CAN 控制器，映射寄存器、初始化 FIFO/邮箱；
 - 比特率与 FD 参数（data_bitrate、采样点、收发延迟补偿）可配置；
 - 每路 CAN 独立**线程化中断**，报文经 SocketCAN 上送；配合 **HPET/RTC** 打时间戳，满足实时同步需求；
 - 与 Linux-RT 协同降低收发抖动。
- **验证：** ip link set can0 up type can bitrate 500000 dbitrate 2000000 fd on → candump / cansend 回环与互通测试。

7.7.3 网口驱动（GMAC ×3，TSN）

- 2K2000 片上集成 **3 个 GMAC（千兆）**；在 7.4 GMAC 基线之上，本项目完成 **GMAC ×3 千兆网口** 的实网驱动开发，其中 **2 个网口完成 TSN 调优**：
 - 3 个千兆网口（RGMII PHY）独立管理，并对 TSN 网口做 **RGMII 收发延时（tx/rx delay）** 校准；
 - 使能 **MSI 中断**（2K2000 GMAC 支持 MSI，细节见上传的《龙芯2K2000处理器用户手册_V1.0 试用版.pdf》），并配合 **NAPI + 中断线程化** 适配 Linux-RT；
 - 启用 **TSN**（时间敏感网络）特性（手册明确支持），保障确定性时延业务；
 - DMA 描述符环与一致性优化，提升吞吐并降低 CPU 占用。
- **验证：** iperf3 双向打流、ping 长稳、TSN 流量整形验证。

7.7.4 扩展串口驱动

- 在 7.2 片内 UART（3× 全功能 + 1× 双线 UART，8250/16550A 兼容，ACPI _HID 匹配）基础上，本项目扩展**多路串口**能力：
 - **片内 UART：**经 SPCR/ACPI 自动枚举，作为控制台与各业务口；
 - **扩展串口：**经 **PCIe+FPGA**（见 7.7.1）或 USB 转接扩展的额外 UART，驱动以标准 tty 设备暴露（如 /dev/ttySx、/dev/ttyFPGA*），复用 8250 核心并叠加板级参数；
 - 支持波特率、RTS/CTS 流控、**RS485 方向自动切换**；
 - 中断线程化，配合实时场景。
- **验证：** microcom / minicom 外回环、RS485 半双工互发。

8. 移植与适配要点

将 BSP 移植到新单板时，按以下清单执行（**本方案以 ACPI 为核心，不编写 DTS**）：

1. **确认 SoC 型号与架构版本**（LoongArch LA64）。

- 2. **编写/裁剪 ACPI 表**：基于参考板 `ls3a5000.asl`，按需增删设备对象（GPIO、LED、按键、I2C、PCIe 窗口），用 `iasl` 编译为 `ls_acpi.tbl`。
- 3. **校验内存与中断拓扑**：MADT 中 CPU/中断控制器条目、MCFG 中 PCIe 配置窗口与实际单板一致。
- 4. **PMON 适配**：更新加载地址、`RSDP_ADDR`、串口引脚、存储器初始化参数。
- 5. **内核配置**：`make loongson_defconfig` 后启用 `CONFIG_ACPI`、`CONFIG_PREEMPT_RT`，裁剪/增配驱动。
- 6. **驱动自测**：逐一对基线驱动（串口 SPCR、GMAC 网络、存储、显示）与本项目开发的板级驱动（PCIe+FPGA 控制、CANFD、扩展串口）做冒烟测试。
- 7. **SMP/多核验证**：`nproc` 应等于物理核数，且经 MADT 正确枚举。
- 8. **实时性验证**：结合 `ftrace` / `perf` 定位长临界区、测量调度时延；运行 `stress-ng`、`memtester` 验证长时间稳定性。

9. 构建与烧录系统

一键构建脚本（示意）：

```
#!/bin/bash
set -e
# 1. 编译 PMON（含 ACPI 装载逻辑）
make -C pmon CROSS_COMPILE=loongarch64-linux-gnu- pmon.bin
# 2. 编译 Linux-RT 内核（已打 PREEMPT_RT 补丁）
make -C linux ARCH=loongarch CROSS_COMPILE=loongarch64-linux-gnu- -j$(nproc)
# 3. 编译 ACPI 表（ASL -> AML）
iasl -tc acpi/ls3a5000.asl # 生成 ls3a5000.aml
# 4. 打包镜像
./scripts/pack_image.sh --bl pmon.bin --kernel Image \
    --acpi ls3a5000.aml --rootfs rootfs.ext4
```

烧录方式：

介质	工具	命令示例
SPI Flash	编程器 / flashrom	<code>flashrom -p internal -w pmon.bin</code>
eMMC	dd / 烧录器	<code>dd if=image.bin of=/dev/mmcblk0</code>
网络	TFTP	Bootloader 下 <code>load tftp://server/image</code>

10. 测试与验证

测试项	方法	通过标准
启动	上电观察串口输出	进入登录提示符，ACPI 表解析无报错
多核	<code>nproc</code> 、 <code>cat /proc/cpuinfo</code>	核数正确（MADT 枚举一致）
内存	<code>memtester 512M 3</code>	无错误
存储	<code>dd</code> 读写 / <code>fio</code>	速率达标、无 I/O 错误
网络	<code>iperf3</code> 、 <code>ping</code>	带宽达标、低丢包（GMAC ×3，TSN 生效）
显示	<code>fb-test</code> / X 会话	画面正常
CANFD	<code>candump</code> / <code>cansend</code> 回环与互通	CAN FD 报文收发正常、无丢帧
扩展串口	<code>microcom</code> / <code>minicom</code> 回环、RS485 互发	收发一致、RS485 方向切换正确
PCIe+FPGA 控制	用户态 <code>ioctl</code> / <code>sysfs</code> 读写扩展接口	寄存器/引脚读写正确、中断响应正常
实时性	<code>ftrace</code> / <code>perf</code> （调度延迟周期采样）	最大时延 < 预期阈值（如 <100μs）
稳定性	<code>stress-ng --timeout 24h</code>	不崩溃、不重启、实时抖动稳定

11. 常见问题排查

现象	可能原因	排查手段
上电无任何串口输出	PMON 未运行 / 串口引脚错误	示波器测 TX；核对 SPCR/cmdline 与波特率
卡在 ACPI 解析阶段	RSDP 地址错 / 表 checksum 失败	<code>acpidump</code> 检查 RSDP； <code>iasl -d</code> 反编译核对
内核 panic: VFS: Unable to mount root	root 设备名/分区错	核对 <code>root=</code> 与分区表
网卡无 link	RGMII 延时 / PHY 复位 / <code>_DSD</code> 缺字段	<code>ethtool -d</code> ；核对 DSDT 中 <code>phy-mode</code>
从核不启动	MADT 缺条目 / IPI 实现问题	加 <code>nosmp</code> 验证主核；查 MADT CPU 拓扑

现象	可能原因	排查手段
PCIe 设备缺失	MCFG 窗口错 / PERST# 时序	<code>lspci</code> 空；核对 MCFG 基址与 REFCLK
实时任务抖动大	中断未线程化 / 关抢占过长	<code>ftrace</code> 查长临界区； <code>perf</code> 采样调度延迟

12. 附录

A. 参考命令速查

```
# 查看 ACPI 表（运行时）
ls /sys/firmware/acpi/tables
acpidump > acpi.log && acpixtract -a acpi.log # 导出各表
iasl -d DSDT.dat # 反编译为 DSDT.dsl

# 查看内核启动日志（关注 ACPI 解析与 RT 抢占）
dmesg | grep -iE "acpi|rt|preempt" | less

# 查看实时调度信息
cat /sys/kernel/realtime # 1 表示 RT 内核生效
uname -a # 含 -rt 后缀

# 交叉编译工具链
loongarch64-linux-gnu-gcc --version
```

B. 推荐资料

- Linux 内核 Documentation/arch/loongarch/ 与 Documentation/acpi/ 官方文档
- Linux-RT (PREEMPT_RT) 项目与对应版本补丁
- 龙芯开源社区 (Loongnix / 龙芯中科) BSP 仓库 (PMON + ACPI)
- ACPI 规范：《ACPI_Spec_xx.pdf》(ACPI Specification，设备表结构与命名空间规范，详见 5.1.4)
- 板级资料：《龙芯 2K2000 处理器用户手册 V1.0 试用版》(芯片手册，第 4 章引用)、《增强型综合电子核心管理卡硬件原理图》(硬件平台参考，第 4 章引用)

C. 版本记录

版本	日期	说明
v1.0	2025	初版，覆盖 BSP 总体架构与关键驱动
v1.1	2025	修订：明确 Linux-RT 实时内核、PMON 引导、ACPI 设备管理机制（替代设备树）
v1.2	2025	修订：依据《龙芯 2K2000 处理器用户手册 V1.0》充实第 4 章（龙架构/LA364 核、Cache 一致性、IPI、分层 I/O 中断、2K2000 资源）
v1.3	2025	修订：新增 7.7「设计开发的关键板级驱动」，体现 PCIe+FPGA 平台扩展控制、CANFD、GMAC ×3 网口（其中 2 口 TSN 调优）、扩展串口四类自研驱动，并同步交付物/测试/移植清单

本报告为通用设计模板，具体单板请以厂商提供的 *BSP 仓库与数据手册* 为准。